

Package ‘mudnester’

October 2, 2026

Title Surveillance Data Cleaning and Preparation for Public Health

Version 0.7.8

Description Clean, prepare, and aggregate surveillance data for public health analysis.

Provides structural data cleaning and standardisation (`clean_the_nest()`), age categorisation against ~50 published schemes with publication-ready labelling (`preening()`), time-unit aggregation with zero-filling and seasonal awareness (`roost()`), joint aggregation of several linked event dates (e.g. onset, admission, ICU, complication, fatality) into one table of comparable rate columns (`flyway()`), under-ascertainment correction via a stratified, time-varying multiplier factor supplied directly, derived by the ratio (multiplier) method, or derived by inverting an externally sourced severity ratio (e.g. an infection-fatality-rate anchor) against an observed severity ratio (`corncrake()`), comorbidity detection from ICD-10-AM clinical coding (`plumage()`), vaccine coverage data construction (`brood()`), hash-based de-identification (`molting()`), and relinking of previously de-identified data (`homing()`). `brood()` produces a `brood_df` object supporting two population models: pre-aggregated denominators (`population_model = ``pre_aggregated"`) and record-level cohort designs (`population_model = ``cohort"`). The cohort model handles single time-point coverage snapshots, interrupted time series analysis via a built-in sweep returning monthly coverage rates (`time_series = TRUE`), and birth cohort designs with person-time computation. This cohort/time-series coverage model was applied in Roughan et al. (2026) [doi:10.33321/cdi.2026.50.031](https://doi.org/10.33321/cdi.2026.50.031) to estimate infant immunisation coverage against respiratory syncytial virus over an 18-month period. Both wide format (one row per person with dose columns, from `'starling'::murmuration()`) and long format (one row per dose) are accepted. `corncrake()` returns both a point-corrected count and uncertainty bounds wherever they can be derived, including the inverse relationship between a severity-anchored factor and the bounds of its own reference rate. Built for Australian public health surveillance practice but not specific to it -- see individual function documentation for notes on non-Australian use (e.g. Northern Hemisphere season boundaries).

License MIT + file LICENSE

Depends R (>= 4.1)

Imports dplyr (>= 1.1.0), tidyr (>= 1.3.0), lubridate (>= 1.9.0),
 stringr (>= 1.5.0), rlang (>= 1.1.0), tibble (>= 3.2.0), digest
 (>= 0.6.30), janitor (>= 2.2.0), utils, stats

Suggests testthat (>= 3.0.0), knitr (>= 1.42), rmarkdown (>= 2.20),
 usethis (>= 2.1.0), gtsummary, ggplot2

Config/testthat/edition 3

Encoding UTF-8

Language en-GB

LazyData true

RoxygenNote 8.0.0

VignetteBuilder knitr

URL <https://github.com/nrsmoll/mudnester>

BugReports <https://github.com/nrsmoll/mudnester/issues>

NeedsCompilation no

Author Nicolas Smoll [aut, cre] (ORCID:
<https://orcid.org/0000-0002-6923-9701>),
 Moderna [fnd] (Support for this package's development was provided via
 the Moderna Global Research Fellowship)

Maintainer Nicolas Smoll <nicolas.smoll@health.qld.gov.au>

Repository CRAN

Date/Publication 2026-10-02 11:50:02 UTC

Contents

age_schemes	3
brood	4
clean_the_nest	10
corncrake	16
flyway	23
homing	26
list_age_schemes	27
molting	28
plumage	30
preening	35
print.brood_df	38
roost	38

Index	43
--------------	-----------

age_schemes

*Age Banding Scheme Definitions***Description**

A tibble of ~50 named age-banding schemes used by [preening\(\)](#) and [list_age_schemes\(\)](#). Each row defines one scheme: its breaks, labels, family, focus tags, source citation, and audit metadata.

Usage

```
age_schemes
```

Format

A tibble with 50 rows and 11 variables:

scheme Character. Unique snake_case scheme identifier.

family Factor. One of "national_stats", "international_stats", "vaccination", "surveillance", "clinical_developmental", "disease_specific".

focus_tags List-column of character vectors. One or more focus tags from the fixed vocabulary: paediatric, aged_care, who_standard, vaccination, surveillance, broad, fine_grained, national_au, research.

n_bands Integer. Number of age bands (length of labels).

breaks List-column of numeric vectors. Break points; always starts at 0, ends at Inf, strictly increasing.

labels List-column of character vectors. Band labels; length equals length(breaks) - 1.

age_unit Factor. "years" or "days".

source Character. Short citation.

source_url Character. URL to the source document (NA if none).

last_checked Date. Date the scheme was last verified against its source.

is_illustrative Logical. TRUE if the scheme follows a documented convention but lacks a single primary citation.

Source

See `vignette("age-schemes", package = "mudnester")` for the full catalogue with source citations. Constructed in `data-raw/age_schemes.R`.

See Also

[preening](#), [list_age_schemes](#)

brood

*Construct a Vaccine Coverage Data Object***Description**

Bird note: A brood is the full clutch under a parent bird's care – every egg counted, every hatchling tracked, none overlooked. The parent knows precisely how many eggs were laid (the eligible population), how many have hatched (the vaccinated), and which ones are still waiting. The gap between the clutch and the hatchlings is not an absence; it is information. `brood()` does the same for a vaccination cohort: it takes the full eligible population and the vaccinated subset, structures them into a validated, self-documenting `brood_df` object, and makes the coverage gap as visible as the coverage itself. The resulting `brood_df` is ready for `bowerbird::brood_plot()`.

Constructs and validates a `brood_df` object for vaccine coverage analysis. Accepts two input formats and two population models:

Input formats:

- **Wide format** (`data_format = "wide"`): one row per person with vaccination dates in columns `vax_date_1` through `vax_date_N` and optionally paired type columns. This is the natural output of `starling::murmuration()`.
- **Long format** (`data_format = "long"`): one row per dose, with `id_col`, `vax_date_col`, and optionally `vaccine_type_col`.

Population models:

1. **Pre-aggregated** (`population_model = "pre_aggregated"`, default): supply a summary data frame with `n_vaccinated` and `n_eligible` already computed per stratum (and optionally per time period via `date_col`). Does not perform per-person dose logic. Accepts `person_time_col` for person-time denominators.
2. **Cohort** (`population_model = "cohort"`): record-level data, one row per person. This is the backbone model. Handles all individual-level designs:
 - **Single time point**: set window and `reference_date` (or `intervention_date`) for a snapshot coverage figure.
 - **Time series** (`time_series = TRUE`): sweeps window = "`current_at_date`" across monthly (or other) intervals between `ts_start` and `ts_end`. Returns one row per stratum per time point, with a `reference_date` column and an `intervention_period` column auto-populated when `intervention_date` is supplied. This is the primary design for interrupted time series analysis of vaccination coverage.
 - **Birth cohort**: a special case of the cohort model. Set `entry_date_col = "dob"` and supply `eligibility_days` and `ensor_date` to get birth-cohort behaviour with person-time computation.

Assessment windows (`population_model = "cohort"`):

- "`current_at_date`" (default): currently covered at `reference_date`. Used automatically for every time point when `time_series = TRUE`.
- "`at_exit`": vaccinated at or before `exit_date_col`.

- "pre_entry": vaccinated before cohort entry (baseline).
- "post_entry_days": vaccinated within window_days days of cohort entry.
- "post_intervention": vaccinated on or after intervention_date.

Usage

```
brood(
  data,
  data_format = "wide",
  population_model = "pre_aggregated",
  stratum_col = "stratum",
  vaccine_type_col = NULL,
  dose_col = NULL,
  date_col = NULL,
  n_vaccinated_col = "n_vaccinated",
  n_eligible_col = "n_eligible",
  person_time_col = NULL,
  vax_date_cols = NULL,
  vax_type_cols = NULL,
  id_col = "id_var",
  vax_date_col = "vax_date",
  vax_target = NULL,
  vax_target_exact = FALSE,
  validity_days = Inf,
  entry_date_col = "entry_date",
  exit_date_col = "exit_date",
  window = "current_at_date",
  window_days = 365L,
  intervention_date = NULL,
  post_window_days = NULL,
  reference_date = NULL,
  eligibility_days = NULL,
  censor_date = NULL,
  time_series = FALSE,
  ts_start = NULL,
  ts_end = NULL,
  ts_by = "month",
  denominator_notes = NULL
)
```

Arguments

data	A data frame. For population_model = "pre_aggregated", a pre-aggregated summary with one row per stratum (or per stratum per time period). For "cohort", a record-level data frame with one row per person (wide format) or one row per dose (long format).
data_format	Character. "wide" (default) or "long". Ignored for population_model = "pre_aggregated".

population_model	Character. "pre_aggregated" (default) or "cohort".
stratum_col	Character or NULL. Stratification column name. Default "stratum". If not found in data, a single "Overall" placeholder is used. When time_series = TRUE, each stratum gets its own row at every time point.
vaccine_type_col	Character or NULL. Vaccine type column for long format or pre-aggregated data. For wide format, use vax_type_cols. Default NULL.
dose_col	Character or NULL. Dose number/label column. Default NULL.
date_col	Character or NULL. Date column for pre-aggregated time-varying input. Default NULL.
n_vaccinated_col	Character. Column containing vaccinated count. Default "n_vaccinated".
n_eligible_col	Character or NULL. Column containing eligible population count. Default "n_eligible".
person_time_col	Character or NULL. Column containing person-time denominator. Default NULL.
vax_date_cols	Character vector. Names of the dose date columns (e.g. paste0("vax_date_", 1:10)). Up to 20 columns supported.
vax_type_cols	Character vector or NULL. Names of paired vaccine type columns. Required if vax_target is supplied. Default NULL.
id_col	Character. Person identifier column. Default "id_var".
vax_date_col	Character. Single dose date column. Default "vax_date".
vax_target	Character vector or NULL. Vaccine type(s) to count. NULL (default) counts any dose regardless of type. Case-insensitive partial match by default (e.g. "Shingrix" matches "Shingrix RZV").
vax_target_exact	Logical. Use exact string matching if TRUE. Default FALSE.
validity_days	Numeric. Days after administration after which a dose is considered expired. Default Inf (no expiry – appropriate for Shingrix, which shows durable protection to at least 10 years). Set to 365 for annual influenza vaccine.
entry_date_col	Character. Cohort entry date column. For birth cohort studies, pass "dob" here and also supply eligibility_days. Required. Default "entry_date".
exit_date_col	Character or NULL. Cohort exit date column. Required for window = "at_exit". Default "exit_date".
window	Character. Coverage assessment anchor for single-timepoint designs. One of "current_at_date" (default), "at_exit", "pre_entry", "post_entry_days", or "post_intervention". Ignored when time_series = TRUE (which always uses "current_at_date" internally).
window_days	Integer. Days after cohort entry for window = "post_entry_days". Default 365L.
intervention_date	Date or NULL. For window = "post_intervention": the fixed date on or after which vaccination is assessed. Required for that window. For time-series mode: used to label rows as "Pre-intervention" or "Post-intervention" in the intervention_period column. Default NULL.

post_window_days	Integer or NULL. Days forward from intervention_date to search for vaccinations (window = "post_intervention" only). NULL uses exit_date_col.
reference_date	Date or NULL. Anchor date for single-timepoint window = "current_at_date". Ignored when time_series = TRUE. Default NULL.
eligibility_days	Integer or NULL. For birth cohort studies: days from entry_date_col (typically DOB) during which a person is eligible. When supplied, per-person exit date is computed as $\min(\text{entry_date} + \text{eligibility_days}, \text{censor_date})$, and person-time (days at risk) is also returned. Set to 180L for a 6-month nirsevimab eligibility window. Default NULL.
censor_date	Date or NULL. Administrative censoring date for birth cohort designs. Persons still within their eligibility window at censor_date contribute person-time only to that date. Default NULL (uses Sys.Date()).
time_series	Logical. If TRUE, sweeps window = "current_at_date" across a sequence of dates from ts_start to ts_end at ts_by intervals, returning one row per stratum per time point. Default FALSE.
ts_start	Date or NULL. Start of the time-series sweep. Required when time_series = TRUE. This is typically set to 1 year before any intervention of interest, to establish a baseline trend. Default NULL.
ts_end	Date or NULL. End of the time-series sweep. Required when time_series = TRUE. Default NULL.
ts_by	Character. Time interval between successive reference dates. One of "week", "month" (default), "quarter", or "year".
denominator_notes	Character or NULL. Free-text denominator description. Stored as a metadata attribute and displayed by print.brood_df() and optionally as a plot caption by bowerbird::brood_plot(). Default NULL.

Value

A brood_df object: a classed data frame with columns:

stratum Stratum label (character).

reference_date Reference date for cohort time-series output (Date). NA for pre-aggregated output.

intervention_period For time-series with intervention_date supplied: "Pre-intervention" or "Post-intervention" (character). NA otherwise.

vaccine_type Vaccine product label (character or NA).

dose_number Dose label (character or NA).

n_vaccinated Vaccinated count, numerator (integer).

n_eligible Eligible population count, denominator (integer or NA).

person_time Person-time denominator (numeric or NA – only populated for birth cohort designs).

coverage Proportion vaccinated, 0-1 scale (numeric).

`window_description` Auto-generated description of the coverage window applied (character).

Metadata attributes:

- `attr(x, "population_model")`: "pre_aggregated" or "cohort"
- `attr(x, "data_format")`: "wide" or "long"
- `attr(x, "time_series")`: TRUE or FALSE
- `attr(x, "time_series_by")`: interval used (time-series only)
- `attr(x, "n_time_points")`: number of time points (time-series)
- `attr(x, "intervention_date_ts")`: intervention date supplied (time-series only, for `bowerbird::brood_plot()` to draw a vertical reference line)
- `attr(x, "window")`: single-timepoint window used
- `attr(x, "validity_days")`: validity window applied
- `attr(x, "vax_target")`: vaccine type filter applied
- `attr(x, "denominator_notes")`: free-text denominator notes
- `attr(x, "n_strata")`: number of unique strata
- `attr(x, "mudnester_version")`: package version
- `attr(x, "created_at")`: POSIXct creation timestamp

Pre-aggregated denominator arguments (`population_model = "pre_aggregated"`)

Used when data is already a stratum-level summary.

Wide format dose arguments

Used when `data_format = "wide"` with record-level data.

Long format dose arguments

Used when `data_format = "long"` with record-level data.

Vaccine targeting

Controls which vaccine type(s) count as valid doses.

Vaccine validity

Controls how long a dose is considered currently protective.

Cohort arguments

Apply when `population_model = "cohort"`.

Time-series arguments

Apply when `population_model = "cohort"` and `time_series = TRUE`. This is the primary mode for interrupted time series analysis of vaccination coverage.

See Also

`bowerbird::brood_plot()` for visualising `brood_df` objects. [roost](#) for aggregating cases and hospitalisations. [clean_the_nest](#) for upstream data cleaning. `starling::murmuration()` for the linkage step that produces the wide-format linked dataset used as input for cohort coverage designs.

Examples

```
# Pre-aggregated coverage (no dose-level data needed)
brood(
  data.frame(stratum = c("0-17", "18-49", "50-64", "65+"),
             n_vaccinated = c(480L, 3550L, 2370L, 1760L),
             n_eligible = c(1000L, 5000L, 3000L, 2000L)),
  denominator_notes = "ABS ERP 2024, SC LGA."
)

# -- Time-series: monthly Shingrix coverage in a JAK-inhibitor cohort -----
# A small stand-in for what starling::murmuration() would return in a real
# pipeline: one row per person, entry_date_nominal plus up to 2 doses.
linked_cohort <- data.frame(
  entry_date_nominal = as.Date(c("2024-01-15", "2024-03-20", "2024-06-10",
                                "2024-09-05", "2025-01-12", "2025-04-18")),
  vax_date_1 = as.Date(c("2024-11-02", NA, "2025-02-14",
                        "2025-01-20", NA, "2025-06-01")),
  vax_type_1 = c("Shingrix", NA, "Shingrix", "Fluvax", NA, "Shingrix"),
  vax_date_2 = as.Date(c(NA, "2025-08-15", NA, NA, NA, NA)),
  vax_type_2 = c(NA, "Shingrix", NA, NA, NA, NA)
)

vax_date_cols <- grep("^vax_date_", names(linked_cohort), value = TRUE)
vax_type_cols <- grep("^vax_type_", names(linked_cohort), value = TRUE)

cov_ts <- brood(
  linked_cohort,
  data_format = "wide",
  population_model = "cohort",
  vax_date_cols = vax_date_cols,
  vax_type_cols = vax_type_cols,
  vax_target = "Shingrix",
  validity_days = Inf,
  entry_date_col = "entry_date_nominal",
  time_series = TRUE,
  ts_start = as.Date("2024-12-01"),
  ts_end = as.Date("2026-06-01"),
  ts_by = "month",
  intervention_date = as.Date("2025-12-01"),
  denominator_notes = "JAK-inhibitor cohort, SC HHS. Shingrix only, no expiry."
)

print(cov_ts)

# -- Single time point: pre-intervention snapshot -----
```

```

cov_pre <- brood(
  linked_cohort,
  data_format      = "wide",
  population_model = "cohort",
  vax_date_cols    = vax_date_cols,
  vax_type_cols    = vax_type_cols,
  vax_target       = "Shingrix",
  validity_days    = Inf,
  entry_date_col   = "entry_date_nominal",
  window           = "current_at_date",
  reference_date   = as.Date("2025-11-30"),
  denominator_notes = "JAK-inhibitor cohort. Pre-intervention Shingrix coverage."
)

# -- Birth cohort: nirsevimab coverage (6-month eligibility window) -----
birth_records <- data.frame(
  dob = as.Date(c("2024-05-03", "2024-06-14", "2024-07-22", "2024-08-30")),
  nirsevimab_date = as.Date(c("2024-05-10", NA, "2024-07-25", "2025-04-01")),
  vax_type_1 = c("nirsevimab", NA, "nirsevimab", "nirsevimab"),
  gestation_category = c("Term", "Preterm", "Term", "Preterm")
)

cov_birth <- brood(
  birth_records,
  data_format      = "wide",
  population_model = "cohort",
  vax_date_cols    = "nirsevimab_date",
  vax_type_cols    = "vax_type_1",
  vax_target       = "nirsevimab",
  entry_date_col   = "dob",
  eligibility_days  = 180L,
  censor_date      = as.Date("2024-12-31"),
  stratum_col      = "gestation_category",
  denominator_notes = "SCPHU birth cohort 2024. Eligible = 0 to 180 days from birth."
)

```

clean_the_nest

Clean and Standardise Surveillance Data

Description

Bird note: The White-winged Chough (*Corcorax melanorhamphos*) — after which mudnester is named — builds its mud nest in meticulous layers, returning again and again to reinforce weak spots before adding anything new on top. `clean_the_nest()` is that foundational layer: nothing downstream is trustworthy until the structure here is sound. Of note, Starlings are cavity nesters, meaning they prefer to build their homes inside existing holes and crevices — the cleaned nest `clean_the_nest()` prepares is what starling: `murmuration()` moves into.

Cleans surveillance dataset types and prepares them for record linkage. This is the first step in the mudnester pipeline. Works with case notification datasets (linelists such as Notifiable Conditions registers), hospitalisation datasets (administrative datasets like iPM), vaccination datasets (such as the Australian Immunisation Register), and generic person-level cohorts (`data_type = "cohort"`) — any linelist of people (birth cohort, study cohort, outbreak linelist, flight manifest) being prepared for linkage but without disease-specific semantics of its own. All datasets should include information suitable for data linkage — first name, last name, date of birth, Medicare number, gender, and/or postcode.

Classic pipeline:

1. `clean_the_nest()` — clean and standardise. Pay close attention to linkage variables (letter-names, dob, Medicare number, gender, postcode).
2. `starling::murmuration()` — link cases to vaccination data (c2v).
3. `starling::murmuration()` — link c2v to hospitalisation data (c2v2h). Vaccination linkage is optional.
4. `preening()` — age categorisation and publication-ready labelling. Works well with `gtsummary::tbl_summary()`.
5. `roost()` — aggregate to a chosen time unit for visualisation.
6. `molting()` — de-identify before sharing or archiving.

Usage

```
clean_the_nest(
  data,
  id_var = NULL,
  event_id_var = NULL,
  drop_eggs = FALSE,
  data_type = NULL,
  lie_nest_flat = FALSE,
  drop_the_na_vax = TRUE,
  keep_vars = NULL,
  diagnosis = NULL,
  lettername1 = NULL,
  lettername2 = NULL,
  dob = NULL,
  age = NULL,
  medicare = NULL,
  postcode = NULL,
  gender = NULL,
  fn = NULL,
  latitude = NULL,
  longitude = NULL,
  onset_date = NULL,
  cohort_entry_date = NULL,
  cohort_exit_date = NULL,
  vax_type = NULL,
  vax_date = NULL,
  lag = 0,
```

```

admission_date = NULL,
discharge_date = NULL,
hospital = NULL,
icd_code = NULL,
diagnosis_description = NULL,
drg = NULL,
icu_date = NULL,
icu_hours = NULL,
dialysis = NULL,
genomics = NULL,
dod = NULL,
died = NULL
)

```

Arguments

data	A data frame. Can be a case notifications dataset (infections), hospital admissions, or vaccination dataset. Dates must be in Date format before calling this function.
id_var	Character. Name of the column uniquely identifying each individual. Critical for linkage — must be non-missing. For "cases", must be unique per row (one row per person, or first infection only). For "hospital" and "vaccination", one person can appear in multiple rows.
event_id_var	Character. Name of the column uniquely identifying each event (hospitalisation or vaccination). Must be unique across the whole dataset. Some datasets have duplicate event IDs — this is checked and raises an error.
drop_eggs	Logical. If TRUE (default FALSE), drops all columns not needed for linkage or downstream analysis, producing a lean dataset. Set FALSE when you need to retain additional context columns.
data_type	Character. One of "cases", "hospital", "vaccination", or "cohort" ("generic" is accepted as a synonym for "cohort"). Required — no default. Age and age categories are calculated from dob and onset_date when both are present (typically "cases" and "hospital"); not calculated for "vaccination". Use "cohort" for a generic linelist of people being prepared for linkage — e.g. a birth cohort, a study cohort, an outbreak or flight-manifest linelist — that has no disease-specific semantics of its own but still needs its linkage variables (letternames, dob, Medicare, gender, postcode) cleaned and standardised before <code>starling::murmuration()</code> . Like "cases", a "cohort" must be one row per person (<code>id_var</code> unique); unlike "cases", it requires no diagnosis or onset_date. Pair with <code>cohort_entry_date/cohort_exit_date</code> to carry the follow-up window through to a <code>cohort_window</code> linkage.
lie_nest_flat	Logical. If TRUE, pivots a long vaccination dataset (one or more rows per person, as from the Australian Immunisation Register) to wide format — one row per person with <code>vax_date_1</code> , <code>vax_date_2</code> , etc. Requires <code>data_type = "vaccination"</code> and <code>id_var</code> , <code>vax_type</code> , <code>vax_date</code> .
drop_the_na_vax	Logical. If TRUE (default), drops vaccination rows where <code>vax_type</code> is NA before pivoting. Only relevant when <code>lie_nest_flat = TRUE</code> .

keep_vars	Character vector of additional column names to retain when drop_eggs = TRUE. Passed to <code>dplyr::any_of()</code> .
diagnosis	Character. Name of the column containing the infectious disease diagnosis (e.g. "COVID-19", "RSV", "Influenza").
lettername1	Character. Name of the first name column. Non-alphanumeric characters are stripped and values lowercased. Middle names are removed (only the first token is kept).
lettername2	Character. Name of the last name column. Non-alphanumeric characters are stripped and values lowercased. Two-part surnames are kept.
dob	Character. Name of the date-of-birth column (must be Date class). Used for record linkage and age calculation. In birth-cohort studies (e.g. nirsevimab or Abrysvo effectiveness), dob is typically also the cohort entry date — pass the same column name to both dob and cohort_entry_date and the aliasing is handled internally.
age	Character. Name of a pre-calculated age column (numeric). Supply only when you do not want age recalculated from dob and onset_date.
medicare	Character. Name of the Medicare number column. Australian Medicare number structure (11 digits total when the IRN is included): • Digits 1–8 — Account identifier: a unique 8-digit number assigned to a household or account. The first digit is always 2–6. • Digit 9 — Checksum: mathematically derived from digits 1–8 to validate the number. Verified by <code>clean_the_nest()</code> when the full 10-digit number is present. • Digit 10 — Card issue number: increments each time a new card is issued (lost, expired, or family member added). Cannot be 0. • Digit 11 / IRN — Individual Reference Number: the single digit printed to the left of a person's name on the card. Up to 9 people can share one Medicare card; the primary cardholder is usually 1, a partner 2, children 3, 4, etc. This digit is often stored separately or appended as the 11th character. Derived output columns: • <code>medicare_clean</code> — digits only, spaces removed, as supplied • <code>medicare08</code> — digits 1–8 (account identifier, for household-level linkage) • <code>medicare09</code> — digits 1–9 (account + checksum; the "base" number used for individual identity validation) • <code>medicare10</code> — digits 1–10 (full card number including issue digit; use for card-specific linkage, e.g. AIR) • <code>medicare_irn</code> — digit 11 / IRN (individual position on the card); NA if fewer than 11 digits are present • <code>medicare_valid</code> — logical; TRUE if the checksum (digit 9) is mathematically correct for digits 1–8
postcode	Character. Name of the postcode column.
gender	Character. Name of the gender column. Values are left as-is (clean to a consistent format before calling, e.g. all "F"/"M").

fn	Character. Name of the First Nations status column.
latitude	Character. Name of the latitude column.
longitude	Character. Name of the longitude column.
onset_date	Character. Name of the onset/diagnosis date column (must be Date class). Age is calculated from dob and onset_date when both are supplied and age is not.
cohort_entry_date	Character. Name of the cohort entry date column (must be Date class). The date an individual formally begins accumulating time-at-risk. Conceptually distinct from dob (linkage/age variable) and onset_date (event date). Used by roost() and starling::murmuration() as an anchor for risk windows. In birth-cohort studies this is the same column as dob — pass the same name to both.
cohort_exit_date	Character. Name of the cohort exit date column (must be Date class). The date an individual leaves the analytic cohort (end of follow-up or administrative censoring).
vax_type	Character. Name of the vaccine type/brand/antigen column.
vax_date	Character. Name of the vaccination event date column (Date class). Arrange rows in chronological order before calling if you want vax_date_1 to be the earliest dose.
lag	Numeric. Days to add to vax_date (e.g. 14 for COVID-19 peak immunity). Default 0.
admission_date	Character. Name of the hospital admission date column (must be Date class).
discharge_date	Character. Name of the discharge date column (must be Date class). Length of stay (los) is derived when both admission_date and discharge_date are supplied.
hospital	Character. Name of the hospital identifier column.
icd_code	Character. Name of the ICD-10-AM code column.
diagnosis_description	Character. Name of the ICD code description column (for human readability; not critical for analysis).
drg	Character. Name of the AR-DRG column.
icu_date	Character. Name of the ICU admission date column (Date class).
icu_hours	Character. Name of the ICU hours column (numeric). ICU days are derived automatically.
dialysis	Character. Name of the dialysis indicator column (0/1).
genomics	Character. Name of the genomics/variant column (e.g. SARS-CoV-2 variant, Hepatitis A genotype).
dod	Character. Name of the date-of-death column (Date class). If both dob and dod are supplied, age_at_death is derived.
died	Character. Name of a pre-existing death indicator column (0/1). When NULL and dod is supplied, death_outcome is derived automatically.

Value

A data frame with standardised column names, derived linkage variables (blocking variables, composite name strings), and domain-specific derived columns (length of stay, age, ICU outcome, death outcome, Medicare variants). If `drop_eggs = TRUE`, retains only linkage and analysis columns.

See Also

[preening](#) for age categorisation after cleaning, [roost](#) for time-unit aggregation, [molting](#) for de-identification before sharing, [homing](#) for relinking de-identified data.

Examples

```
# Case notifications
case_data <- data.frame(
  identity = 1:2,
  first_name = c("Alex", "Sam"),
  surname = c("Nguyen", "Patel"),
  dob = as.Date(c("1990-02-14", "1985-11-03"))
)
df_diag <- clean_the_nest(
  case_data, data_type = "cases", drop_eggs = TRUE,
  id_var = "identity",
  lettername1 = "first_name",
  lettername2 = "surname",
  dob = "dob"
)

# Hospital admissions
hosp_data <- data.frame(
  patient_id = 1:2,
  firstname = c("Alex", "Sam"),
  last_name = c("Nguyen", "Patel"),
  birth_date = as.Date(c("1990-02-14", "1985-11-03")),
  admission_date = as.Date(c("2025-01-10", "2025-02-20"))
)
df_hosp <- clean_the_nest(
  hosp_data, data_type = "hospital", drop_eggs = TRUE,
  id_var = "patient_id",
  lettername1 = "firstname",
  lettername2 = "last_name",
  dob = "birth_date",
  admission_date = "admission_date"
)

# Vaccination records -- pivot one-row-per-dose data to one row per person
vax_data <- data.frame(
  patient_id = c(1, 1, 2),
  firstname = c("Alex", "Alex", "Sam"),
  last_name = c("Nguyen", "Nguyen", "Patel"),
  vax_type = c("Influenza", "COVID-19", "Influenza"),
  vax_date = as.Date(c("2025-04-01", "2025-04-01", "2025-04-15"))
)
```

```

df_vax <- clean_the_nest(
  vax_data, data_type = "vaccination", lie_nest_flat = TRUE,
  id_var      = "patient_id",
  lettername1 = "firstname",
  lettername2 = "last_name",
  vax_type    = "vax_type",
  vax_date    = "vax_date"
)

# Birth-cohort study (nirsevimab / Abrysvo effectiveness):
# dob is the cohort entry date -- pass the same column name to both.
birth_cohort_data <- data.frame(
  baby_id      = 1:2,
  first_name   = c("Ivy", "Leo"),
  last_name    = c("Chen", "Brown"),
  babys_date_of_birth = as.Date(c("2024-03-01", "2024-04-15")),
  end_of_followup_date = as.Date(c("2024-09-01", "2024-10-15"))
)
df_cohort <- clean_the_nest(
  birth_cohort_data, data_type = "cohort",
  id_var      = "baby_id",
  lettername1 = "first_name",
  lettername2 = "last_name",
  dob         = "babys_date_of_birth",
  cohort_entry_date = "babys_date_of_birth",
  cohort_exit_date  = "end_of_followup_date"
)

```

corncrake

Correct Surveillance Counts for Under-Ascertainment

Description

Bird note: Corncrakes are famously detected far more often by ear than by eye – the overwhelming majority of records are calls, not sightings. Field surveyors have long applied call-based correction factors to convert a count of detections into an estimate of the true, largely-unheard population behind it. `corncrake()` does the same to a surveillance count: it takes an observed count, together with a factor that may vary by stratum and by time, and returns an estimate of the true total sitting behind the ascertained one.

Applies a stratified, time-varying ascertainment (multiplier) factor to a count column, appending the resolved factor and a corrected count to the data – with uncertainty bounds wherever they can be derived. Designed to receive output from [roost](#) (default `count_col` = "n" matches directly, and `time_col` auto-detects from a `roost_tbl`), but works on any tidy data frame with a count column.

Factor sources (method):

- **User-supplied** (method = "user_supplied", default): supply `factor_table`, a lookup table of ascertainment factors that may vary by `group_by` stratum and by a `date_start/date_end`

validity window. This is the appropriate choice when factors come from a published estimate, an external evaluation study, or expert judgement.

- **Ratio estimate** (method = "ratio_estimate"): the factor is derived internally as $\text{secondary_count} / \text{count}$ at each stratum/time point, using a second, more-complete data stream supplied via `secondary_data` and `secondary_count_col` (passed through `...`). This is the classic surveillance "multiplier method" – e.g. dividing total positive laboratory tests by notified cases to estimate under-notification.
- **Severity anchor** (method = "severity_anchor"): the factor is derived by comparing an *observed* severity ratio in data – e.g. a case-fatality or case-hospitalisation rate, computed as $\text{severity_count_col} / \text{count_col}$ – against a `reference_rate` representing the believed-true rate from a well-ascertained source (e.g. an externally published infection- fatality rate). This is the case-fatality/infection-fatality-rate anchor inversion described in Smoll et al.'s Queensland COVID-19 under-ascertainment analysis ($UAF = CFR_{obs} / IFR_{ref}$): under-ascertainment inflates the observed severity ratio above the true one, and their ratio directly recovers the ascertainment factor. It requires no seroprevalence survey, which is its main advantage early in a novel outbreak (WHO's "Disease X" scenario) before any other anchor is available. `reference_rate` (passed through `...`) can be a single scalar (with optional `reference_rate_lower/reference_rate_upper` for uncertainty) or a `factor_table`-shaped lookup with a rate column (optionally `rate_lower/rate_upper`) for a stratified or time-varying reference rate. **The relationship is inverse:** because the factor is *divided* by the reference rate, a *higher* reference rate produces a *lower* factor. Bounds therefore cross over – the factor's lower bound is computed from the reference rate's *upper* bound, and vice versa – which `corncrake()` handles internally so it doesn't need to be worked out by hand for every use.
- **Known UAF** (method = "known_uaf"): the *reverse* direction of "severity_anchor". Rather than deriving the under-ascertainment factor, you *supply* it – borrowed from a published estimate, another jurisdiction, or a prior "severity_anchor" run – and `corncrake()` applies it to recover the true infection burden and corrected severity rates:

$$N_{true} = N_{obs} \times UAF$$

$$cCHR = \text{hospitalisations} / N_{true}$$

$$cCFR = \text{deaths} / N_{true}$$

Supply `uaf` (through `...`) as a single positive number (with optional `uaf_lower/uaf_upper`) or a `factor_table`-shaped data frame with a `factor` column for a stratified or time-varying UAF (e.g. the quarterly UAFs from the Queensland analysis). Supply `severity_count_col` and/or `death_count_col` (through `...`) to also compute `cCHR/cCFR`; omit both to get `n_true` only. This is the natural companion to "severity_anchor": one derives a UAF from a reference rate, the other applies a known UAF to get corrected rates – together they cover the "Disease X" preparedness workflow of borrowing another jurisdiction's `cCFR/cCHR` to estimate your own UAF and true burden, or vice versa. As with "severity_anchor", the `cCHR/cCFR` bounds invert (a larger N_{true} gives a smaller corrected rate), which `corncrake()` handles internally.

Usage

```
corncrake(
```

```

data,
count_col = "n",
method = "user_supplied",
factor_table = NULL,
group_by = NULL,
time_col = NULL,
on_missing = "warn_na",
ci_method = "table",
denominator_col = NULL,
rate_multiplier = 1e+05,
...
)

```

Arguments

<code>data</code>	A data frame, typically the output of roost .
<code>count_col</code>	Character. Name of the observed count column. Default "n" (matches roost 's output directly).
<code>method</code>	Character. "user_supplied" (default), "ratio_estimate", "severity_anchor", or "known_uaf". See Description.
<code>factor_table</code>	A data frame of ascertainment factors. Required for <code>method = "user_supplied"</code> ; ignored (with a warning if supplied) for <code>method = "ratio_estimate"</code> . Required columns: <code>date_start</code> , <code>date_end</code> (Date, or coercible; NA means an open/unbounded end), and <code>factor</code> (positive numeric). Optional: <code>factor_lower</code> , <code>factor_upper</code> (numeric uncertainty bounds), <code>source</code> (character citation, carried through to <code>attr(x, "ascertainment_source")</code>). If <code>group_by</code> is supplied, <code>factor_table</code> must also contain those stratifying columns, with non-overlapping <code>date_start/date_end</code> windows within each stratum.
<code>group_by</code>	Character vector of stratifying column names present in both <code>data</code> and <code>factor_table</code> (e.g. "age_group" from preening – values are matched as character strings, so label spelling must match exactly). NULL (default) applies a single global, time-varying-only factor to every row.
<code>time_col</code>	Character or NULL. Name of the date column in <code>data</code> used to look up the time-varying factor. NULL (default) auto-detects the aggregation column from <code>attr(data, "roost_meta")</code> when <code>data</code> is a <code>roost_tbl</code> ; otherwise it must be supplied explicitly. Must be coercible to Date – <code>roost_tbl</code> objects built with <code>time_unit = "epiweek"</code> , "season", or "season_year" are not, and need a separate Date column supplied here instead.
<code>on_missing</code>	Character. Behaviour when a row has no matching ascertainment factor for its stratum/time: "warn_na" (default, a single <code>warning()</code> naming the count of affected rows, which are left NA) or "error".
<code>ci_method</code>	Character. How <code>corrected_count_lower/corrected_count_upper</code> are derived: "table" (default; uses <code>factor_table</code> 's <code>factor_lower/factor_upper</code> against the observed count as given), "propagate" (treats the observed count as Poisson-distributed and propagates its exact 95% interval through the point factor – useful when the factor itself is treated as fixed but the underlying count is

understood to be a realisation of a random process), or "none" (point estimate only).

denominator_col Character or NULL. Optional population denominator column in data. If supplied, `corrected_rate(+_lower/_upper)` columns are also computed as `corrected_count / denominator * rate_multiplier`.

rate_multiplier Numeric. Multiplier used for `corrected_rate`. Default 100000. Ignored if `denominator_col` is NULL.

... For `method = "ratio_estimate"`: `secondary_data` (a data frame sharing the same `time_col` and `group_by` column names as `data`) and `secondary_count_col` (character; default matches `count_col`). For `method = "severity_anchor"`: `severity_count_col` (character; the observed severity outcome count column in data, at the same stratification/time as `count_col` – e.g. deaths or hospitalisations), `reference_rate` (a single positive number, or a `factor_table`-shaped data frame with a rate column), `reference_rate_lower/reference_rate_upper` (numeric; only used when `reference_rate` is a scalar – ignored with a warning if `reference_rate` is a table, which should carry its own `rate_lower/rate_upper` instead), and `reference_source` (character; citation for the reference rate, appended to `attr(x, "ascertainment_source")`). For `method = "known_uaf"`: `uaf` (a single positive number, or a `factor_table`-shaped data frame with a factor column for a stratified/time-varying UAF), `uaf_lower/uaf_upper` (numeric; only used when `uaf` is a scalar), `uaf_source` (character citation), and optionally `severity_count_col` and/or `death_count_col` (character; observed hospitalisation and death count columns in data, used to compute `cCHR/cCFR`).

Value

`data` with the following columns appended, and the same class as the input (so a `roost_tbl` stays a `roost_tbl` – see [roost](#)):

ascertainment_factor The resolved point factor for that row.

ascertainment_factor_lower, ascertainment_factor_upper From `factor_table`, if supplied; from the reference rate's own bounds for `method = "severity_anchor"` (inverted – see Description); NA for `method = "ratio_estimate"`, which derives a point estimate only.

corrected_count `count_col * ascertainment_factor`.

corrected_count_lower, corrected_count_upper Per `ci_method`; NA if `ci_method = "none"` or unavailable.

corrected_rate* Only if `denominator_col` supplied.

n_true, n_true_lower, n_true_upper Only for `method = "known_uaf"`: the corrected true infection count ($N_{obs} \times UAF$) and its bounds – identical values to `corrected_count`, exposed under the manuscript's name.

cCHR, cCHR_lower, cCHR_upper Only for `method = "known_uaf"` when `severity_count_col` is supplied: corrected case-hospitalisation rate ($hospitalisations/N_{true}$), with inverted bounds.

cCFR, cCFR_lower, cCFR_upper Only for `method = "known_uaf"` when `death_count_col` is supplied: corrected case-fatality rate ($deaths/N_{true}$), with inverted bounds.

Metadata attributes `attr(x, "ascertainment_method")`, `attr(x, "ascertainment_source")`, and `attr(x, "corncrake_meta")` (a list: `method`, `count_col`, `time_col`, `group_by`, `ci_method`, `on_missing`, `n_rows`, `n_missing`, `mudnester_version`, `created_at`) are attached so `bowerbird::roost_plot()` can auto-label a corrected line without extra arguments.

References

Smoll N, Carroll H, Andrews R, Lambert SB, Khandaker G. Queensland's COVID-19 surveillance: a Bayesian Markov chain Monte Carlo analysis. Manuscript in submission, *Eurosurveillance* (2026). The "severity_anchor" and "known_uaf" methods implement the two directions of the under-ascertainment-factor relationship used in this analysis ($UAF = CFR_{obs}/IFR_{ref}$; $N_{true} = N_{obs} \times UAF$; corrected cCHR/cCFR = $severity/N_{true}$). Note the manuscript's *core* estimation is a full seroprevalence-anchored Bayesian MCMC model (implemented in Stan), which `corncrake()` deliberately does **not** reimplement – these methods are the lightweight, closed-form anchor and its inverse, suitable for "Disease X" preparedness and for reusing published UAF/severity estimates from other jurisdictions, not a substitute for the full Bayesian model.

See Also

[roost](#) to build the aggregated counts `corncrake()` corrects. [flyway](#) to build a single table with several linked event counts (e.g. cases and deaths together), directly usable as `severity_count_col` for `method = "severity_anchor"`. [preening](#) for the `age_group` (or other) stratification labels typically used in `group_by`.

Examples

```
# -- Stratified + time-varying, user-supplied factors -----
cases <- data.frame(
  month      = as.Date(c("2024-01-01", "2024-02-01", "2024-01-01", "2024-02-01")),
  age_group  = c("0-17", "0-17", "18+", "18+"),
  n          = c(10L, 14L, 40L, 55L)
)

factors <- data.frame(
  age_group   = c("0-17", "0-17", "18+", "18+"),
  date_start  = as.Date(c("2024-01-01", "2024-02-01", "2024-01-01", "2024-02-01")),
  date_end    = as.Date(c("2024-01-31", "2024-02-29", "2024-01-31", "2024-02-29")),
  factor      = c(2.5, 2.2, 1.4, 1.3),
  factor_lower = c(1.9, 1.7, 1.1, 1.0),
  factor_upper = c(3.3, 2.9, 1.8, 1.7),
  source      = "Illustrative multiplier, SCPHU surveillance evaluation 2025"
)

corncrake(
  cases,
  count_col = "n",
  factor_table = factors,
  group_by   = "age_group",
  time_col   = "month"
)
```

```

# Chained straight after roost() -- time_col auto-detected as "month"
df_diag <- data.frame(
  onset_date = as.Date(c("2024-01-05", "2024-01-20", "2024-02-02",
                        "2024-02-14", "2024-01-10", "2024-02-25")),
  age_group = c("0-17", "0-17", "18+", "18+", "0-17", "18+")
)
cases_monthly <- roost(df_diag, date_col = "onset_date", time_unit = "month",
                      group_cols = "age_group")
cases_corrected <- corncrake(
  cases_monthly,
  factor_table = factors,
  group_by = "age_group"
)

# Ratio (multiplier) method: derive the factor from a more-complete
# secondary stream (e.g. all positive laboratory tests) instead of
# supplying one directly
lab_positive_monthly <- data.frame(
  age_group = c("0-17", "0-17", "18+", "18+"),
  month = as.Date(c("2024-01-01", "2024-02-01", "2024-01-01", "2024-02-01")),
  n = c(25L, 30L, 90L, 110L)
)
cases_corrected2 <- corncrake(
  cases_monthly,
  method = "ratio_estimate",
  group_by = "age_group",
  secondary_data = lab_positive_monthly,
  secondary_count_col = "n"
)

# Severity anchor: early in a novel ("Disease X") outbreak, before any
# seroprevalence survey exists, invert an externally published
# infection-fatality rate against locally observed deaths and cases --
# requires flyway(), not roost(), since both a case count and a death
# count are needed together
df_linked <- data.frame(
  onset_date = as.Date(c("2025-01-05", "2025-01-20", "2025-02-02",
                        "2025-02-14", "2025-03-01", "2025-03-10")),
  fatality_date = as.Date(c(NA, NA, "2025-02-10", NA, NA, NA)),
  age_group = c("18-64", "65+", "65+", "18-64", "65+", "18-64")
)
linked_monthly <- flyway(
  df_linked,
  events = c(cases = "onset_date", deaths = "fatality_date"),
  group_cols = "age_group"
)
cases_corrected3 <- corncrake(
  linked_monthly,
  count_col = "n_cases",
  method = "severity_anchor",
  group_by = "age_group",
  severity_count_col = "n_deaths",

```

```

reference_rate      = 0.01, # reference IFR, e.g. from WHO or a
                        # high-ascertainment reference jurisdiction
reference_rate_lower = 0.005,
reference_rate_upper = 0.020,
reference_source     = "WHO Disease X planning scenario, IFR 1.0% (0.5-2.0%)"
)

# Known UAF (reverse direction): a UAF has already been estimated (here, the
# cumulative Queensland 2022 value of 2.98) -- apply it to recover the true
# infection count and the corrected severity rates from observed counts.
obs <- data.frame(
  quarter      = as.Date(c("2022-01-01", "2022-04-01",
                           "2022-07-01", "2022-10-01")),
  n            = c(670083L, 432457L, 246464L, 80652L), # notified cases
  hospitalisations = c(6902L, 5478L, 5093L, 4278L),
  deaths       = c(734L, 578L, 699L, 318L)
)
qld_corrected <- corncrake(
  obs,
  count_col      = "n",
  method         = "known_uaf",
  time_col       = "quarter",
  uaf            = 2.98, # cumulative UAF (95% CrI 2.78-3.29)
  uaf_lower      = 2.78,
  uaf_upper      = 3.29,
  uaf_source     = "Smoll et al. 2026, Queensland COVID-19 cumulative UAF",
  severity_count_col = "hospitalisations",
  death_count_col  = "deaths"
)
# -> adds n_true, cCHR (+bounds), cCFR (+bounds)

# A time-varying UAF is supplied the same way, as a factor_table-shaped
# `uaf` with a `factor` column (e.g. the manuscript's quarterly UAFs).
quarterly_uaf <- data.frame(
  date_start     = as.Date(c("2022-01-01", "2022-04-01",
                             "2022-07-01", "2022-10-01")),
  date_end       = as.Date(c("2022-03-31", "2022-06-30",
                             "2022-09-30", "2022-12-31")),
  factor         = c(2.35, 2.60, 3.10, 4.08),
  factor_lower   = c(2.08, 2.30, 2.60, 2.41),
  factor_upper   = c(2.68, 3.00, 3.90, 6.38)
)
qld_corrected_tv <- corncrake(
  obs,
  count_col      = "n",
  method         = "known_uaf",
  time_col       = "quarter",
  uaf            = quarterly_uaf,
  severity_count_col = "hospitalisations",
  death_count_col  = "deaths"
)

```

Description

Bird note: A flyway is the established migratory corridor linking a bird population's successive stopover and staging sites – the same birds, traced through each waypoint of a single journey. `flyway()` does the same for a linked case cohort: it traces the same population through however many clinical milestones matter (onset, admission, ICU, complication, death), aggregating each waypoint separately by its own date, and folding the results into one table aligned by time unit and stratum.

Calls `roost` once per date column named in `events`, then joins the results on the shared time-unit and `group_cols` columns, producing one row per stratum/time point with one count column per event, named `n_` followed by that event's name (e.g. `n_cases`). This is the natural next step after a case-to-hospital (or similar) linkage via `starling::murmuration()`, where a single linked record carries several milestone dates for the same person – e.g. to compute an observed case-hospitalisation rate as `n_hospitalisations / n_cases` directly from one table, or to feed `n_hospitalisations/n_deaths` into `corncrake`'s `method = "severity_anchor"`.

Implemented as a thin wrapper, not a change to `roost` itself: each event's own calendar-grid zero-filling, NA-date handling, and validation is exactly `roost`'s own, unchanged. A stratum/time point present for one event but absent for another (e.g. no deaths yet recorded in a stratum where cases already exist) is zero-filled after the join, consistent with `roost`'s own zero-filling philosophy.

Usage

```
flyway(
  data,
  events,
  time_unit = "month",
  group_cols = NULL,
  week_start = 1,
  hemisphere = c("southern", "northern"),
  warn_na = TRUE,
  anchor_event = NULL
)
```

Arguments

<code>data</code>	A data frame of linked surveillance records, with one date column per milestone event.
<code>events</code>	A named character vector (or list) mapping short event names to date column names in <code>data</code> – for example, naming just two events, <code>c(cases = "onset_date", deaths = "fatality_date")</code> . Output count columns are named <code>n_</code> followed by the event name (so that example produces <code>n_cases</code> and <code>n_deaths</code>). If unnamed, columns are named <code>n_event_1</code> , <code>n_event_2</code> , ... instead (with a message suggesting names for clarity).

<code>time_unit</code>	Passed through to every underlying <code>roost</code> call – the same <code>time_unit</code> is used for every event, which is what makes the resulting count columns directly comparable (e.g. dividing one by another to get a rate). See <code>roost</code> .
<code>group_cols</code>	Passed through to every underlying <code>roost</code> call – the same <code>group_cols</code> is used for every event. See <code>roost</code> .
<code>week_start</code>	Passed through to every underlying <code>roost</code> call. See <code>roost</code> .
<code>hemisphere</code>	Passed through to every underlying <code>roost</code> call. See <code>roost</code> .
<code>warn_na</code>	Passed through to every underlying <code>roost</code> call. See <code>roost</code> .
<code>anchor_event</code>	NULL (default) for the original independent-per- event bucketing described above. Or a single event name (one of the names in events) to switch to case-linked aggregation instead – every other event is then bucketed by THIS event's own date. Use this whenever the resulting counts will be divided into a rate (a case-hospitalisation, case-fatality, or similar ratio), especially over a period narrower than roughly a year; the default mode remains appropriate for a reference/context epicurve where each event's own real-world timing (e.g. hospital admission volume by admission date, for bed-planning) is what's actually wanted.

Value

A `flyway_tbl` object (which also inherits `roost_tbl`, so it works directly with `corncrake`'s `time_col` auto-detection and with the existing `roost_tbl` `print/subset` methods): a tibble with any `group_cols`, optionally `epiyear`, the named time-unit column, and one `n_`-prefixed count column per entry in events. Metadata is attached as `attr(x, "roost_meta")` (matching `roost`'s own shape) and `attr(x, "flyway_meta")` (per-event detail, including each event's own `roost_meta`, and – when set – which event `anchor_event` named).

Two aggregation modes – independent vs. case-linked (`anchor_event`)

By default (`anchor_event = NULL`), every event is bucketed by ITS OWN date, independently – `n_cases` in a given period counts onsets falling in that period; `n_hospitalisations` counts admissions falling in that same period, a separate population, not "of the cases whose onset was in this period, how many were hospitalised". Since a downstream milestone (admission, ICU, death) can fall some days or weeks after the case's own onset, this is a good approximation summed over a long enough period – but it can produce an impossible rate (a hospitalised count exceeding the case count) over a narrow window, whenever admission volume in a nearby period doesn't track onset volume in the window being looked at. This is a real, confirmed failure mode, not a hypothetical one – see the `anchor_event` example below.

Setting `anchor_event` (naming one entry in events, typically the case/ onset event) switches to case-linked aggregation instead: every OTHER event is bucketed by the ANCHOR's own date, not its own – "of the rows whose anchor fell in this period, how many also have a non-NA value in this other event's date column". A case and its own downstream hospitalisation/ICU/death are then always attributed to the same period, whichever period the milestone itself actually fell in, which is what makes the resulting counts a valid ratio (e.g. `n_hospitalisations / n_cases`) even over a single narrow period, not just when summed over a long one. The anchor event's own row-level accounting (calendar-grid building, zero-filling, NA-dropping) is still exactly `roost`'s own, applied to the anchor's date column alone.

See Also

[roost](#), which `flyway()` calls once per event. [corncrake](#), which can consume `flyway()` output directly – e.g. `count_col = "n_cases"`, `method = "severity_anchor"`, `severity_count_col = "n_deaths"`.

Examples

```
# After a case-to-hospital linkage via starling::murmuration(), a single
# linked record carries several milestone dates for the same person
linked_cohort <- data.frame(
  onset_date      = as.Date(c("2025-01-05", "2025-01-20", "2025-02-02",
                              "2025-02-14", "2025-03-01", "2025-03-10")),
  admission_date  = as.Date(c("2025-01-08", NA, "2025-02-05",
                              NA, "2025-03-03", NA)),
  icu_date        = as.Date(c(NA, NA, "2025-02-07", NA, NA, NA)),
  complication_date = as.Date(c(NA, NA, NA, NA, "2025-03-06", NA)),
  fatality_date   = as.Date(c(NA, NA, "2025-02-10", NA, NA, NA)),
  age_group       = c("18-64", "65+", "65+", "18-64", "65+", "18-64")
)
linked_monthly <- flyway(
  linked_cohort,
  events = c(
    cases      = "onset_date",
    hospitalisations = "admission_date",
    icu        = "icu_date",
    complications = "complication_date",
    deaths     = "fatality_date"
  ),
  time_unit = "month",
  group_cols = "age_group"
)

# Observed (uncorrected) case-hospitalisation rate, directly from one table
linked_monthly$chr_obs <- linked_monthly$n_hospitalisations / linked_monthly$n_cases

# anchor_event: the same call, but computing a rate over a narrow window
# (the most recent 12 months, say) needs case-linked aggregation, not the
# default independent bucketing above -- without anchor_event, a burst of
# admissions linked to cases whose own onset fell just before the window
# can push n_hospitalisations above n_cases for that same window, an
# impossible rate.
linked_monthly_linked <- flyway(
  linked_cohort,
  events = c(
    cases      = "onset_date",
    hospitalisations = "admission_date",
    icu        = "icu_date",
    complications = "complication_date",
    deaths     = "fatality_date"
  ),
  time_unit = "month",
  group_cols = "age_group",
```

```

    anchor_event = "cases"
  )

```

homing

Relink De-identified Data Using the molting() Lookup Table

Description

Bird note: Homing pigeons are renowned for their extraordinary ability to return to their loft from any location — navigating hundreds of kilometres to find their way back to the place they started. `homing()` does the same thing for a de-identified dataset: it takes records that have been stripped of their identifiers by `molting` and navigates them back to the original identities, using the lookup table as its internal compass. The path home is only available to those who hold the lookup.

Relinks de-identified data with original identifiers using the lookup table created by `molting`. The hash column present in both the de-identified data and the lookup table is the join key. Typically called only when authorised re-identification is required (e.g. clinical follow-up, audit).

Usage

```

homing(
  deidentified_data,
  lookup_table,
  hash_col_name = "row_hash",
  keep_hash = TRUE
)

```

Arguments

<code>deidentified_data</code>	A de-identified data frame containing a hash column. Typically <code>molting()\$deidentified</code> .
<code>lookup_table</code>	The lookup table data frame mapping hashes back to original identifiers. Typically <code>molting()\$lookup</code> . Store this file securely and separately from <code>deidentified_data</code> .
<code>hash_col_name</code>	Character. Name of the hash column used as the join key. Must exist in both <code>deidentified_data</code> and <code>lookup_table</code> . Default <code>"row_hash"</code> .
<code>keep_hash</code>	Logical. If <code>TRUE</code> (default), retains the hash column in the relinked output. Set <code>FALSE</code> to remove it after relinking.

Value

A data frame with the original identifiers merged back into the de-identified data. The class and column order of `deidentified_data` are preserved, with identifier columns appended on the right (or left of the hash column if `keep_hash = FALSE`).

See Also

[molting](#) to create the de-identified data and lookup table, [clean_the_nest](#) for the upstream data preparation step.

Examples

```
# Create sample data and de-identify it (same data molting()'s own
# example uses, so the two paired examples stay consistent)
patient_data <- data.frame(
  patient_name = c("John Doe", "Jane Smith"),
  dob          = as.Date(c("1980-01-01", "1975-05-15")),
  mrn          = c("12345", "67890"),
  age5cat      = factor(c("18-64", "18-64")),
  diagnosis    = c("Condition A", "Condition B"),
  lab_value    = c(120, 95)
)
result <- suppressMessages(molting(patient_data))

# Relink when authorised follow-up is needed
relinked <- homing(
  deidentified_data = result$deidentified,
  lookup_table     = result$lookup
)

# Drop the hash column from the final relinked dataset
relinked_clean <- homing(
  result$deidentified,
  result$lookup,
  keep_hash = FALSE
)
```

list_age_schemes

List Available Age Banding Schemes

Description

Bird note: see `preening()` — this is `preening()`'s companion lookup function, a quick way to see every "arrangement" of feathers (age schemes) on offer before picking one, narrowed down to just the arrangements relevant to the task at hand.

Prints (and invisibly returns) a data frame of age schemes available to `preening()`: scheme name, family, focus tags, age range covered, number of bands, and source citation. By default lists all ~50 schemes; supply family, focus, and/or max_bands to return only a relevant subset — e.g. just the paediatric-focused schemes, or just the WHO-standard ones — instead of scrolling through all 50 every time.

Usage

```
list_age_schemes(family = NULL, focus = NULL, max_bands = NULL)
```

Arguments

family	Optional character vector filter, one or more of "national_stats", "international_stats", "vaccination", "surveillance", "clinical_developmental", "disease_specific". NULL (default) includes all families.
focus	Optional character vector filter on cross-cutting focus tags, e.g. "paediatric", "who_standard", "aged_care", "vaccination", "fine_grained", "broad". A scheme matches if it carries <i>any</i> of the requested tags. NULL (default) includes all foci.
max_bands	Optional integer. Only include schemes with at most this many age bands. NULL (default) applies no limit.

Value

Invisibly, a tibble with columns scheme, family, focus (list-column of tags), n_bands, age_range, source. Printed to console by default, sorted by family then scheme name.

Examples

```
list_age_schemes()                # all ~50
list_age_schemes(focus = "paediatric")  # paediatric-focused only
list_age_schemes(focus = "who_standard")  # WHO standard schemes only
list_age_schemes(family = "surveillance", max_bands = 6)  # compact surveillance schemes
```

molting

De-identify a Dataset with Hash-Based Relinking

Description

Bird note: Moulting is how a bird sheds its identifying plumage — distinctive breeding colours, worn feathers, recognisable patterns — and replaces it with something uniform and anonymous. `molting()` does the same to a dataset: it strips the identifying columns and replaces them with a cryptographic hash for each row, so the bird (patient) is still distinguishable by its new uniform feathers, but cannot be named from them. The old plumage is kept safely in a lookup table, which [homing](#) uses to re-dress the bird when authorised relinking is needed.

Removes personally identifiable information (PII) from a data frame using column-name pattern matching, creates a cryptographic row hash from the removed identifiers, and returns the de-identified data alongside a secure lookup table for later relinking via [homing](#). Age category variables (age2cat, age5cat, etc.) are automatically preserved as they are not directly identifying.

Security note: The lookup table contains sensitive information and must be stored securely with appropriate access controls, separate from the de-identified dataset. Consider encrypting the lookup file before archiving.

Usage

```
molting(
  data,
  id_cols = NULL,
  pii_patterns = NULL,
  additional_pii_cols = NULL,
  hash_method = "sha256",
  hash_col_name = "row_hash",
  return_lookup = TRUE,
  seed = NULL
)
```

Arguments

<code>data</code>	A data frame to de-identify.
<code>id_cols</code>	Optional character vector of column names to use for hashing. If <code>NULL</code> (default), uses the PII columns detected automatically by <code>pii_patterns</code> .
<code>pii_patterns</code>	Optional character vector of regular expression patterns used to detect PII columns for removal. Defaults to a standard list covering common identifiers (name, address, phone, email, dob, mrn, etc.).
<code>additional_pii_cols</code>	Optional character vector of specific column names to remove as PII in addition to those detected by pattern matching. Useful for dataset-specific identifiers without modifying patterns.
<code>hash_method</code>	Character. Hashing algorithm. One of "sha256" (default, recommended), "md5", "sha1", "sha512", "crc32", "xxhash32", "xxhash64", "murmur32", "spookyhash", or "blake3". See <code>?digest::digest</code> for details. SHA-256 balances collision resistance and speed for most surveillance datasets.
<code>hash_col_name</code>	Character. Name for the new hash column. Default "row_hash".
<code>return_lookup</code>	Logical. If <code>TRUE</code> (default), returns a list with both the de-identified data and the lookup table. If <code>FALSE</code> , returns only the de-identified data frame (irreversible — use with care).
<code>seed</code>	Optional integer seed for reproducible hashing with seeded algorithms. Default <code>NULL</code> .

Value

If `return_lookup = TRUE` (default): a named list with elements `$deidentified` (de-identified data frame with hash column) and `$lookup` (the lookup table mapping hash to removed identifier columns). If `return_lookup = FALSE`: only the de-identified data frame.

See Also

[homing](#) to relink de-identified data using the lookup table, [clean_the_nest](#) to prepare and standardise data before de-identification, [roost](#) to aggregate data (aggregated counts can often be shared without de-identification).

Examples

```
# Create sample data
patient_data <- data.frame(
  patient_name = c("John Doe", "Jane Smith"),
  dob          = as.Date(c("1980-01-01", "1975-05-15")),
  mrn          = c("12345", "67890"),
  age5cat      = factor(c("18-64", "18-64")),
  diagnosis    = c("Condition A", "Condition B"),
  lab_value    = c(120, 95)
)

# Basic de-identification – age categories automatically retained
result <- suppressMessages(molting(patient_data))
names(result$deidentified)

# Use MD5 for a shorter hash
result_md5 <- suppressMessages(molting(patient_data, hash_method = "md5"))

# Return only de-identified data (no lookup – irreversible)
deidentified_only <- suppressMessages(molting(patient_data, return_lookup = FALSE))

# Specify exactly which columns to hash
result_ids <- suppressMessages(molting(patient_data, id_cols = c("mrn", "dob")))
```

plumage

Identify Chronic Comorbidities Using ICD-10-AM U-Codes and Acute Admission Codes

Description

Bird note: A bird's plumage is the outward, visible sign of what's underneath – condition, age, health, even diet all show up in feather quality and colour to a practised observer. `plumage()` does the same for a hospitalisation record: the ICD-10-AM codes on file are the visible markings, and this function reads them to surface the chronic conditions underneath, turning a code list into a visible comorbidity profile.

Analyses a hospitalisation dataset to identify chronic conditions based on both ICD-10-AM U-codes (supplementary chronic condition codes) and principal/secondary acute admission ICD-10-AM codes.

The dual-code strategy is necessary because patients are frequently admitted under an acute diagnosis code (e.g., J44.1 for COPD exacerbation) without a supplementary U-code being recorded. Using only U-codes will undercount comorbidity burden, and will systematically miss exactly the admissions where the condition is the ACTIVE reason for the encounter rather than a background comorbidity noted in passing – see `active_inactive` below.

Usage

```
plumage(
  df,
  icd_column,
  prefix = NULL,
  decimal = TRUE,
  drop_eggs = FALSE,
  include_drg = FALSE,
  drg_column = NULL,
  active_inactive = "both"
)
```

Arguments

<code>df</code>	A data frame containing hospitalisation records.
<code>icd_column</code>	Character string specifying the name of the column containing ICD-10-AM codes. Multiple codes per row should be separated by any non-alphanumeric character (comma, space, semicolon, pipe, etc.).
<code>prefix</code>	Optional character string to prefix all output column names. Default is NULL (no prefix).
<code>decimal</code>	Logical. Should codes be matched with decimal points (TRUE, default) or without (FALSE)? When TRUE, matches "U78.1" and "J44.1" format; when FALSE, matches "U781" and "J441" format.
<code>drop_eggs</code>	Logical. If TRUE, drop individual binary condition columns and retain only summary count columns and the category factor. Default is FALSE.
<code>include_drg</code>	Logical. If TRUE, the function also searches for AR-DRG codes (e.g., "E65A", "E65B") in addition to ICD codes. Requires either a separate <code>drg_column</code> argument or that DRG codes are present in <code>icd_column</code> . Default is FALSE because DRG codes are typically stored in a separate column.
<code>drg_column</code>	Optional character string specifying a separate column containing AR-DRG codes. Used only when <code>include_drg = TRUE</code> . Default is NULL.
<code>active_inactive</code>	Character string: one of "both" (default), "active", or "inactive". Chooses which signal populates each condition's single output column – this does NOT add extra columns, it changes what the existing <condition> column means: • "both" (default, and the only behaviour previous versions of this function had): 1 if EITHER the U-code OR an acute ICD-10-AM/AR-DRG code was found. • "active": 1 only if an acute ICD-10-AM code or AR-DRG code was found – the condition was the active/acute reason for, or a significant factor in, this admission. The U-code is ignored. • "inactive": 1 only if the U-code was found – a background/historical comorbidity, regardless of whether an acute code is also present. The acute ICD-10-AM/AR-DRG codes are ignored. Note: for any condition whose <code>icd_stems</code> is empty and <code>include_drg</code> is FALSE (or that condition also has no <code>drg_codes</code>), <code>active_inactive = "active"</code> will flag nobody for that

condition – there is currently no acute-code definition to match against for it.

Details

Code Strategy:

Each chronic condition is detected by searching for **any** of the following that appear in the specified column(s):

1. **U-code** — ICD-10-AM supplementary chronic condition code (e.g., U83.2 for COPD). This is the **inactive** signal: it marks the condition as a known comorbidity, without indicating it was the reason for this particular admission.
2. **Acute ICD-10-AM code** — Principal or secondary admission code that unambiguously indicates the chronic condition (e.g., J44 codes for COPD, E84 for cystic fibrosis). This is the **active** signal: it marks the condition as clinically active in this admission.
3. **AR-DRG code** (optional) — Australian Refined Diagnosis Related Group code (e.g., E65A/E65B for COPD), searched only when `include_drg = TRUE`. Also counted as an **active** signal.

A patient is flagged for a condition (the `<condition>` column) according to `active_inactive`: "both" (default, and the only behaviour previous versions of this function had) if **any** matching code is detected; "active" for the acute ICD-10-AM/AR-DRG signal alone; "inactive" for the U-code signal alone. See that argument's description above.

`icd_stems` (the acute-code definitions) were originally populated only for the Respiratory conditions (plus cystic fibrosis). This revision adds `icd_stems` for the remaining 23 conditions, using standard ICD-10-AM chapter-level codes, so that `active_inactive = "active"` is meaningful for all conditions listed below, not only the respiratory ones – confirmed accurate. No AR-DRG codes were added for these 23 – `drg_codes` remains empty for all of them, so `include_drg` still only adds signal for the original six respiratory/CF conditions.

Conditions Detected:

Metabolic / Endocrine:

- Obesity — U78.1; E66
- Cystic fibrosis — U78.2; E84, E84.0–E84.9 (also counted in Respiratory)

Mental Health:

- Dementia — U79.1; F00–F03, G30
- Schizophrenia — U79.2; F20
- Depression — U79.3; F32, F33
- Intellectual/developmental disability — U79.4; F70–F73, F78, F79

Neurological:

- Parkinson's disease — U80.1; G20
- Multiple sclerosis — U80.2; G35
- Epilepsy — U80.3; G40
- Cerebral palsy — U80.4; G80
- Paralysis — U80.5; G81, G82, G83

Cardiovascular:

- Ischaemic heart disease — U82.1; I20–I25

- Heart failure — U82.2; I50
- Hypertension — U82.3; I10, I11, I12, I13, I15

Respiratory:

- Emphysema — U83.1; J43, J43.0–J43.9, J98.2, J98.3
- COPD — U83.2; J44, J44.0–J44.9; DRG E65 (root; any complexity split)
- Asthma/Chronic bronchitis — U83.3; J41, J41.0–J41.8, J42, J45, J45.0–J45.9, J46; DRG E69 (root; any complexity split)
- Bronchiectasis — U83.4; J47; DRG E77 (root; any complexity split)
- Chronic respiratory failure — U83.5; J96.1, J96.10–J96.19
- Cystic fibrosis — U78.2; E84, E84.0–E84.9; DRG E60 (root; any complexity split)

Gastrointestinal:

- Crohn's disease — U84.1; K50
- Ulcerative colitis — U84.2; K51
- Liver failure — U84.3; K72

Musculoskeletal:

- Rheumatoid arthritis — U86.1; M05, M06
- Osteoarthritis — U86.2; M15–M19
- Systemic lupus erythematosus — U86.3; M32
- Osteoporosis — U86.4; M80, M81

Renal:

- Chronic kidney disease — U87.1; N18

Congenital:

- Spina bifida — U88.1; Q05
- Down syndrome — U88.2; Q90

ICD-10-AM Code Matching:

Acute ICD-10-AM codes are matched at the **category level** using anchored regex. For example, specifying J44 matches J44, J44.0, J44.1, J44.8, and J44.9 but does NOT match J440 or J449 when decimal = TRUE (the default). When decimal = FALSE, the same block-level match applies without a decimal separator.

Value

The input data frame with additional columns appended:

Individual condition indicators (unless drop_eggs = TRUE): one binary (0/1) column per condition, optionally prefixed. active_inactive controls what "1" means for each of these columns (see that argument's description above) – it does not add any extra columns.

Summary columns:

- total_conditions — sum of all individual condition flags
- total_metabolic_conditions
- total_mental_health_conditions
- total_neurological_conditions


```
# --- Example 5: active vs inactive comorbidity status ---
hospital_data5 <- data.frame(
  patient_id = 1:3,
  icd_codes = c(
    "N18.5",          # CKD as the acute reason for admission -> active only
    "U87.1",          # CKD noted as a background comorbidity -> inactive only
    "N18.5, U87.1"    # both present
  )
)
results5_both    <- plumage(hospital_data5, "icd_codes") # active_inactive = "both" (default)
results5_active  <- plumage(hospital_data5, "icd_codes", active_inactive = "active")
results5_inactive <- plumage(hospital_data5, "icd_codes", active_inactive = "inactive")
data.frame(
  patient_id = 1:3,
  both       = results5_both$kidney_disease,
  active     = results5_active$kidney_disease,
  inactive   = results5_inactive$kidney_disease
)
```

preening

Age Categorisation and Variable Labelling

Description

Bird note: Preening is how a bird sorts every feather into one of a small set of functional positions, re-arranging the same plumage to suit the moment without changing the bird underneath. `preening()` does the same to a population: the same raw age column is re-sorted into whichever of the ~50 standard schemes an analysis calls for.

Optional ad hoc utility for age categorisation and publication-ready variable labelling. Can be applied at any point in the pipeline — before or after `roost()`. Draws on a library of ~50 named age-banding schemes spanning Australian and international statistical standards, vaccination/clinical guidance, surveillance-system conventions, and disease-specific research bands (see `vignette("age-schemes")` for the full referenced list). `roost()` does not require `preening()` to have been called first.

Users are not required to know all ~50 scheme names. `scheme` can be a single exact name, OR left NULL and narrowed instead with `family`, `focus`, and/or `max_bands` — the same filters exposed by `list_age_schemes()` — so a paediatric-only analysis, say, only ever sees paediatric-relevant schemes rather than all 50.

Usage

```
preening(
  data,
  age_col = NULL,
  scheme = NULL,
  family = NULL,
  focus = NULL,
  max_bands = NULL,
```

```

    age_unit = "years",
    age_breaks = NULL,
    age_labels = NULL,
    label_vars = NULL,
    label_style = "figure"
  )

```

Arguments

<code>data</code>	A data frame at any stage of processing.
<code>age_col</code>	Name of numeric age-in-years column (or age-in-days for neonatal/infant schemes — see <code>age_unit</code>). <code>NULL</code> to skip age banding entirely (e.g. when only <code>label_vars</code> relabelling is wanted).
<code>scheme</code>	Name of a single age banding scheme to apply, e.g. <code>"atagi_covid19_2025"</code> , or <code>"custom"</code> . Leave <code>NULL</code> (default) to instead select via <code>family</code> / <code>focus</code> / <code>max_bands</code> — see Details. Run <code>list_age_schemes()</code> to browse all available scheme names with a one-line description and source citation for each.
<code>family</code>	Optional character vector to filter candidate schemes by family before selection — one or more of <code>"national_stats"</code> , <code>"international_stats"</code> , <code>"vaccination"</code> , <code>"surveillance"</code> , <code>"clinical_developmental"</code> , <code>"disease_specific"</code> . Ignored (with a warning) if scheme is supplied.
<code>focus</code>	Optional character vector to filter candidate schemes by cross-cutting focus tag, e.g. <code>"paediatric"</code> , <code>"who_standard"</code> , <code>"aged_care"</code> , <code>"vaccination"</code> , <code>"fine_grained"</code> , <code>"broad"</code> . A scheme matches if it carries <i>any</i> of the requested tags. Can be combined with <code>family</code> (AND logic between the two filters). Ignored (with a warning) if scheme is supplied.
<code>max_bands</code>	Optional integer. Restrict candidates to schemes with at most this many age bands. Ignored (with a warning) if scheme is supplied.
<code>age_unit</code>	Unit of the values in <code>age_col</code> : <code>"years"</code> (default) or <code>"days"</code> (required for neonatal/early-infancy schemes with sub-annual bands).
<code>age_breaks</code>	Numeric vector of custom break points (only used if <code>scheme = "custom"</code>).
<code>age_labels</code>	Character vector of custom labels (length must equal <code>length(age_breaks) - 1</code> ; only used if <code>scheme = "custom"</code>).
<code>label_vars</code>	Character vector of column names to apply publication-ready labels to. Independent of age banding — can be used with or without <code>age_col</code> .
<code>label_style</code>	<code>"figure"</code> (compact) or <code>"table"</code> (verbose). Only affects <code>label_vars</code> relabelling.

Details

Selecting a scheme — three ways:

1. **Exact:** `preening(data, age_col = "age", scheme = "atagi_covid19_2025")` — applies that one scheme directly, no filtering involved.
2. **Filtered, single match:** `preening(data, age_col = "age", family = "vaccination", focus = "paediatric")` — if exactly one scheme matches the filter, it is applied automatically (with a `message()` naming which scheme was selected, so the choice is never silent).

3. **Filtered, multiple matches:** if more than one scheme matches the filter, `preening()` does not guess — it `stop()`s and prints the matching scheme names so the caller can pick one explicitly via `scheme`.

Call `list_age_schemes()` for the full catalogue, or with the same `family` / `focus` / `max_bands` filters to preview what a given filter combination would match before calling `preening()`.

Value

Input data frame with new or relabelled columns appended. Original columns are always preserved. If age banding was performed, a new `age_group` column is added as an ordered factor, and the scheme's source citation is attached as `attr(x, "age_scheme_source")`.

Examples

```
df <- data.frame(age = c(2, 8, 15, 35, 62, 80))

# Exact scheme
preening(df, age_col = "age", scheme = "atagi_covid19_2025")

# All paediatric-focused schemes, regardless of family -- errors with a
# printed list of matches, since several paediatric schemes exist. Wrapped
# in try() here so the error (the point of this example) doesn't fail the
# example check itself.
try(preening(df, age_col = "age", focus = "paediatric"))

# Narrow further: paediatric AND vaccination family -- confirmed against a
# real check run to still match 2 schemes (pneumococcal_program,
# rsv_maternal_infant), not 1 as filters alone might suggest -- also
# wrapped in try() for the same reason, then resolved by supplying scheme
# explicitly, using one of the two names the error itself lists
try(preening(df, age_col = "age", family = "vaccination", focus = "paediatric"))
preening(df, age_col = "age", scheme = "rsv_maternal_infant")

# WHO-standard schemes only -- wrapped defensively as well, since whether
# this resolves to one match or several depends on the real age_schemes
# data and hasn't been independently confirmed the way the case above was
try(preening(df, age_col = "age", focus = "who_standard"))

# Custom breaks, bypassing the scheme library entirely -- no dependency on
# age_schemes data at all, so no ambiguity is possible here
preening(df, age_col = "age", scheme = "custom",
         age_breaks = c(0, 18, 65, Inf),
         age_labels = c("0-17", "18-64", "65+"))
```

<code>print.brood_df</code>	<i>Print Method for brood_df Objects</i>
-----------------------------	--

Description

Print Method for brood_df Objects

Usage

```
## S3 method for class 'brood_df'
print(x, ...)
```

Arguments

<code>x</code>	A brood_df object produced by <code>brood()</code> .
<code>...</code>	Additional arguments passed to <code>print.data.frame</code> .

Value

Invisibly returns `x`.

<code>roost</code>	<i>Build an Epicurve: Aggregate Surveillance Records by Time Unit</i>
--------------------	---

Description

Bird note: A roost is where a flock settles at the end of the day — many individual birds resolving into one countable gathering at a shared site and time. `roost()` does exactly the same thing to records: many individual rows resolve into counts, grouped by whichever time unit matters for the analysis — day, ISO week, fortnight, month, quarter, year, epidemiological week, or season.

`roost()` is the epicurve builder underneath almost everything on the surveillance side of the aviary ecosystem. Anywhere you'd draw an epicurve — case incidence, hospitalisations, ICU admissions, deaths, vaccine doses delivered — `roost()` is the function turning line-list rows into the counts that curve is built from. It fills zero-count periods explicitly (so epi curves never skip a quiet week) and applies hemisphere-aware season and season-year labels. Designed to receive output from `clean_the_nest` and optionally `preening` (though fully standalone — neither is required), producing a `roost_tbl` ready for `bowerbird::roost_plot()`.

For a single cohort that needs tracing through several downstream milestone dates at once (onset, admission, ICU, death) into one aligned table, see `flyway`, `roost()`'s own wrapper for exactly that shape of problem (Example 2 below) — the natural next step after a `starling::murmuration()` linkage, and the way to get case, hospitalisation, ICU, and death counts into one data frame for a severity cascade or feeding `corncrake`.

Usage

```
roost(
  data,
  date_col,
  time_unit = "month",
  event_cols = NULL,
  group_cols = NULL,
  week_start = 1,
  hemisphere = c("southern", "northern"),
  warn_na = TRUE
)
```

Arguments

data	A data frame of surveillance records, typically the output of clean_the_nest or preening .
date_col	Character. The name of the date column (quoted, e.g. <code>date_col = "onset_date"</code>). Must be coercible to Date.
time_unit	Character. Aggregation unit. One of "day", "isoweek", "fortnight", "month", "biannual", "quarter", "year", "epiweek", "season", or "season_year". Default "month".
event_cols	Character vector of binary (0/1) column names to sum alongside the row count. Use for outcomes such as ICU admission or death. NULL (default) counts rows only.
group_cols	Character vector of stratification column names (e.g. <code>c("age_group", "pathogen")</code>). NULL (default) for unstratified counts. A full calendar grid is built across all combinations of <code>group_cols</code> values, so zero-count strata are retained.
week_start	Integer 1–7 (1 = Monday, 7 = Sunday). Sets the first day of the ISO week when <code>time_unit = "isoweek"</code> . Default 1.
hemisphere	One of "southern" (default — the aviary ecosystem's home convention; Summer = Dec–Feb) or "northern" (Summer = Jun–Aug). Set "northern" for non-Australian surveillance data. Only affects <code>time_unit = "season" or "season_year"</code> — irrelevant, and safe to leave untouched, for every other <code>time_unit</code> . See "Worldwide use" below.
warn_na	Logical. Warn when rows with NA in <code>date_col</code> are dropped. Default TRUE.

Value

A `roost_tbl` object: a tibble with columns for any `group_cols`, optionally `epiyear` (when `time_unit = "epiweek"`), the named time-unit column, `n` (row count), and any `event_cols`. Zero-count periods are filled with 0. Metadata is attached as `attr(x, "roost_meta")`.

Worldwide use, Australian home turf

`roost()` and the rest of the aviary ecosystem work on surveillance data from anywhere — there's nothing Australia-specific in the aggregation logic itself. The defaults, age schemes, and worked

examples are tuned for Australian public health practice (Southern Hemisphere seasons by default, ATAGI-aligned age bands in [preening](#), jurisdictional datasets like AIR and NNDSS in the surrounding tooling) because that's the practice this ecosystem was built inside of. Set hemisphere = "northern" for time_unit = "season" or "season_year" elsewhere, and everything else applies unchanged.

See Also

[clean_the_nest](#) to prepare data before aggregation, [preening](#) to add age groups before stratified aggregation, [flyway](#) to aggregate several milestone dates for the same cohort into one table, [corncrake](#) to correct a roost_tbl's counts for under-ascertainment, [molting](#) to de-identify the aggregated output before sharing.

Examples

```
# -----
# Example 1: Yearly RSV case counts
# A straightforward epicurve -- confirmed RSV notifications aggregated to
# yearly counts for a multi-year burden-of-disease summary.
# -----
rsv_cases <- data.frame(
  notification_date = as.Date(c("2022-04-12", "2022-07-03", "2023-05-20",
                                "2023-08-14", "2024-04-30", "2024-09-02"))
)
rsv_yearly <- roost(rsv_cases, date_col = "notification_date", time_unit = "year")
rsv_yearly

# -----
# Example 2: Influenza, 3 seasons, severity cascade via flyway()
# Confirmed influenza cases across three seasons, linked (via
# starling::murmuration()) to hospitalisation, ICU, and death dates.
# flyway() runs roost() once per milestone and joins the results into one
# table -- the shape a severity cascade or n_hospitalisations/n_cases
# observed-CHR calculation needs.
# -----
influenza_linked_cohort <- data.frame(
  onset_date      = as.Date(c("2023-06-10", "2023-07-22", "2024-06-05",
                              "2024-08-14", "2025-06-20", "2025-07-30")),
  admission_date  = as.Date(c("2023-06-13", NA, "2024-06-08",
                              NA, "2025-06-23", NA)),
  icu_date        = as.Date(c(NA, NA, "2024-06-10", NA, NA, NA)),
  fatality_date   = as.Date(c(NA, NA, "2024-06-15", NA, NA, NA))
)
influenza_by_season <- flyway(
  influenza_linked_cohort,
  events = c(
    cases           = "onset_date",
    hospitalisations = "admission_date",
    icu             = "icu_date",
    deaths          = "fatality_date"
  ),
  time_unit = "season_year"
```

```

)
influenza_by_season$chr_obs <-
  influenza_by_season$n_hospitalisations / influenza_by_season$n_cases

# -----
# Example 5: Disease X, multiple time units and age bands via preening()
# The same case data viewed two ways: a fine-grained weekly curve for
# outbreak monitoring, and a coarser monthly curve stratified by a
# preening()-derived age band for a routine surveillance report.
# -----
disease_x_cases <- data.frame(
  onset_date = as.Date(c("2025-01-05", "2025-01-12", "2025-01-20",
                        "2025-02-02", "2025-02-14", "2025-02-25")),
  age_years = c(5, 34, 68, 12, 45, 80)
)
disease_x_cases <- preening(disease_x_cases, age_col = "age_years",
                           scheme = "atagi_covid19_2025")

disease_x_weekly <- roost(disease_x_cases, date_col = "onset_date",
                        time_unit = "isoweek")

disease_x_monthly_by_age <- roost(disease_x_cases, date_col = "onset_date",
                                time_unit = "month",
                                group_cols = "age_group")

# -----
# Example 3: RSV hospitalisations only, for an interrupted time series
# Monthly RSV-associated hospitalisation counts in infants, stratified by
# era (pre/post a nirsevimab programme start date) -- straight into
# bowerbird::roost_plot(facet_by = "era") for a pre/post comparison panel.
# -----
rsv_infant_admissions <- data.frame(
  admission_date = as.Date(c("2023-05-02", "2023-06-18", "2023-07-30",
                            "2024-05-14", "2024-06-22", "2024-08-05")),
  era = c("Pre-nirsevimab", "Pre-nirsevimab", "Pre-nirsevimab",
          "Post-nirsevimab", "Post-nirsevimab", "Post-nirsevimab")
)
rsv_hosp_monthly <- roost(
  rsv_infant_admissions,
  date_col = "admission_date",
  time_unit = "month",
  group_cols = "era"
)

# -----
# Example 4: Measles vaccine doses delivered during a rollout
# Total measles vaccine doses administered per week during an outbreak-
# response rollout -- the delivery-tempo curve, not a coverage/eligibility
# question (that's brood()'s job -- see brood()'s own examples).
# -----
measles_vax_records <- data.frame(
  vax_date = as.Date(c("2025-03-03", "2025-03-05", "2025-03-10",
                      "2025-03-12", "2025-03-17", "2025-03-19"))
)

```

```
)  
measles_doses_weekly <- roost(  
  measles_vax_records,  
  date_col = "vax_date",  
  time_unit = "isoweek"  
)
```

Index

* datasets

age_schemes, 3

age_schemes, 3

brood, 4, 38

clean_the_nest, 9, 10, 27, 29, 38–40

corncrake, 16, 23–25, 38, 40

flyway, 20, 23, 38, 40

homing, 15, 26, 28, 29

list_age_schemes, 3, 27

molting, 15, 26, 27, 28, 40

plumage, 30

preening, 3, 15, 18, 20, 35, 38–40

print.brood_df, 38

roost, 9, 15, 16, 18–20, 23–25, 29, 38